

Monte Carlo Option Pricing Engine — Methodology & Validation

This document summarizes the methods implemented in this project, the mathematics behind each, and the validation performed to confirm correctness. All results below use the following standard test parameters unless otherwise noted:

```
S0 = 100      (spot price)
K  = 100      (strike price)
r  = 0.05     (risk-free rate)
sigma = 0.20  (volatility)
T  = 1.0      (time to expiry, years)
```

All Monte Carlo prices are reported with 95% confidence intervals ($\pm 1.96 \times$ standard error) unless stated otherwise. Every technique below is cross-validated against at least one independent reference (a closed-form solution, a different numerical method, or an internal consistency check) — no pricer in this project is trusted on its own output alone.

1. Core Simulation Engine

Stock price paths are simulated under the risk-neutral measure using Geometric Brownian Motion (GBM), discretized as:

$$S(t+dt) = S(t) \cdot \exp((r - \sigma^2/2) \cdot dt + \sigma \sqrt{dt} \cdot Z), \quad Z \sim N(0,1)$$

Simulation uses (r) , not the real-world expected return (μ) , since Monte Carlo option pricing relies on risk-neutral valuation — the price is a discounted expectation under the measure where the underlying's expected return equals the risk-free rate.

Validation: simulated terminal prices were checked against the theoretical property $(E[S_T] = S_0 \cdot e^{(rT)})$, and path plots were visually confirmed to show realistic dispersion consistent with the specified volatility.

2. Black-Scholes Closed Form (Ground Truth)

Used as the validation reference for every European-style pricer in this project:

```
d1 = [ln(S0/K) + (r + sigma^2/2) * T] / (sigma*sqrt(T))
d2 = d1 - sigma*sqrt(T)
Call = S0 * N(d1) - K * e^(-r*T) * N(d2)
```

At-the-money reference (S0=K=100): Call = 10.4506, Put = 5.5735

A lesson worth documenting: at-the-money tests can mask bugs

During development, a parenthesization bug in the $\frac{d1}{\sigma\sqrt{T}}$ formula caused the numerator's $\ln(S_0/K)$ term to be excluded from the division by $\sigma\sqrt{T}$. Because the test parameters used $S_0 = K = 100$, $\ln(S_0/K) = 0$, and the bug had **no effect on the output** — every at-the-money validation passed despite the formula being wrong. The bug was only caught when testing at $S_0=100, K=110$, where the buggy formula gave **5.4524** against the correct **6.0401** — a ~10% error. This is now a permanent regression test in the automated suite (`test_bs_off_atm_regression`), and the broader lesson — **validation should always include at least one off-ATM case** — is reflected throughout this project's test design.

3. Variance Reduction Techniques

Four independent methods were implemented and compared on a fair basis (equal path count N, standard error as the metric) for pricing a vanilla European call.

Method	Mechanism	Result at N=100,000
Naive MC	Baseline	SE = 0.04669
Antithetic variates	Pair each draw Z with -Z	SE = 0.03311 → 1.93x variance reduction
Control variate	Correct using known $E[S_T]$	SE = 0.01773
Sobol QMC (RQMC)	Low-discrepancy 1D sequence, randomized-scramble error estimation	SE = 0.00005 → 971x speedup vs. naive

Antithetic variates

For every draw Z, a mirrored path using -Z is also simulated. Since option payoffs are monotonic in the terminal price, paired payoffs are negatively correlated, reducing variance at no extra simulation cost. Validated: 1.93x variance reduction at N=10,000, price unchanged within CI (variance reduction should never bias the estimate, only its precision).

Control variates

Uses the simulated terminal price S_T as a control, since $E[S_T] = S_0 \cdot e^{(rT)}$ is known exactly. The optimal correction coefficient β is estimated from the sample covariance between the payoff and S_T .

Sobol QMC — and the effective-dimensionality insight

An initial implementation simulated the full 252-step path using a 252-dimensional Sobol sequence, which showed **no improvement over naive MC** (identical convergence slope).

This is because Sobol sequences lose their low-discrepancy advantage in high dimensions.

The fix relied on a key mathematical observation: a **vanilla European option's payoff depends only on the terminal price S_T** , which is itself a single Gaussian draw (the sum of all per-step Brownian increments collapses to one draw by the properties of Brownian motion). The problem's *effective dimensionality* is 1, not 252. Re-implementing Sobol sampling directly in 1D (rather than simulating the full path) let QMC's low-discrepancy advantage actually apply, producing a dramatic improvement.

A second issue was found and fixed: the standard $\text{std}/\sqrt{N}$ error formula assumes i.i.d. samples, which Sobol sequences are not by construction (they are deliberately correlated to fill space evenly). Applying that formula to Sobol output understated its true error and made Sobol appear no better than naive MC even after the dimensionality fix. The correct approach — **randomized QMC (RQMC)**: running multiple independently-scrambled Sobol sequences and taking the standard deviation of the resulting price estimates across runs — revealed Sobol's genuine near- $O(1/N)$ convergence, versus $O(1/\sqrt{N})$ for the other three methods.

Note on scope: this 1D reduction is only valid for payoffs depending solely on S_T . Path-dependent payoffs (Asian, barrier, lookback, American) require the full path and would need a Brownian bridge construction to recover any of Sobol's dimensional advantage — noted as a possible extension, not implemented here.

4. Greeks Estimation

Three independent Monte Carlo estimators were implemented for Delta and Vega, all validated against Black-Scholes closed-form Greeks.

Greek	BS (analytical)	Pathwise	Likelihood Ratio	Finite Difference
Delta	0.6368	0.6374 ± 0.0036	0.6303 ± 0.0090	0.6345
Vega	37.5240	37.6540 ± 0.4668	38.2451 ± 1.6834	37.4718

All four methods agree within noise at $N=100,000$. Notably, **likelihood ratio's confidence intervals are ~2.5x (Delta) to ~3.6x (Vega) wider than pathwise's** at equal sample size — consistent with LR's known higher-variance tradeoff. Pathwise differentiates the payoff function directly (works well for smooth payoffs but fails for discontinuous ones, e.g. digital options); LR differentiates the underlying probability density instead (higher variance, but remains valid even where pathwise breaks down). Finite difference used common random numbers (same Z draws for the bumped and base runs) to control its own noise.

5. Exotic Payoffs

Path-dependent payoffs where Monte Carlo is not just an alternative to Black-Scholes but genuinely necessary, since no general closed-form solution exists.

Asian options (average-price)

- **Geometric average** has a closed form (Kemna-Vorst). MC vs. closed form: **5.5520 ± 0.0478 vs. 5.5468** — matched within CI, small residual gap consistent with discrete-vs-continuous averaging.
- **Arithmetic average** (the economically relevant case, no closed form): **5.8045 ± 0.0497**. By Jensen's inequality, the arithmetic mean of a positive series is always $\geq$ its geometric mean, so the arithmetic-average price should exceed the geometric-average price for a call — confirmed ($5.8045 > 5.5520$).

Lookback options (floating strike)

Payoff uses the path's running min (call) or max (put) rather than a fixed strike. Validated: lookback call (16.6389) priced well above the vanilla call (10.4251), consistent with the lookback holder capturing the best possible exercise point along the entire path.

Barrier options (up/down, in/out — all four variants)

Validated two independent ways:

1. **In/out parity** — a knock-in and knock-out option with identical parameters always sum to the vanilla price, since every path either breaches the barrier or doesn't. Confirmed: up-and-out (3.5540) + up-and-in (6.8573) = 10.4112, vs. vanilla 10.4465 (within combined MC noise).
2. **Extreme-barrier limits** — with an unreachable barrier ($B=1000$), up-and-out converged to the vanilla price (10.4622 vs. 10.4465) and up-and-in converged to exactly 0.0000, as expected.

Known limitation: the barrier and running min/max checks are only evaluated at discrete simulation timesteps (252/year), not continuously. This causes discrete monitoring to systematically understate the true breach probability relative to continuous monitoring — a well-documented effect in the literature, correctable via the Broadie-Glasserman-Kou barrier shift, which was not implemented here but is noted as a known simplification.

6. American Options

Longstaff-Schwartz (regression-based early exercise)

At each timestep, walking backward from expiry, in-the-money paths are used to regress their discounted future cash flows against a degree-2 polynomial of the current price, estimating the continuation value. Exercise occurs when immediate exercise value exceeds this estimate.

Two theoretical properties were used as validation:

1. **American $\geq$ European.** An American option has every right a European option has, plus early-exercise optionality, so its price must be at least as high. Confirmed for a put: American 6.0637 vs. European 5.5893 — a real, non-noise early-exercise premium.
2. **American call = European call (no dividends).** Early exercise is never optimal for a call on a non-dividend-paying stock, so the two prices should coincide. Confirmed: American 10.4731 ± 0.0914 vs. European 10.3876 — consistent within noise, versus the clear ~ 0.47 gap seen for the put case.

Binomial tree (independent cross-check)

A Cox-Ross-Rubinstein binomial lattice was built as a second, structurally independent pricer (no simulation, no regression — exact backward induction on a discrete lattice) for American options. LS put price (6.0513 ± 0.0435) and binomial put price (6.0896) agreed within ~ 0.9 standard errors — a genuinely strong cross-validation, since the two methods share no common machinery.

A debugging note worth keeping

Initial testing of `test_american_call_equals_european` failed on two separate pytest runs with an *identical* price difference (0.2738) both times. This looked at first like a systematic bias in the LS estimator — plausible in principle, since LS is documented to have a small upward bias from using the same paths to both fit and evaluate the regression (Longstaff & Schwartz, 2001). Running the same comparison across multiple explicit random seeds, however, showed the difference changing sign and magnitude (+0.0966, -0.0338, +0.0792) — disproving the systematic-bias hypothesis. The real cause was that the test suite lacked explicit RNG seeding in most files, so repeated `pytest` runs were replaying the same deterministic sequence of "random" draws due to fixed test execution order, not genuine randomness. Fixed by explicitly seeding every test file. This is a good illustration of why a repeatable result isn't automatically a *correct* one — the multi-seed check was necessary to distinguish a real structural bias from an artifact of insufficient randomization.

7. Performance

Path simulation: NumPy vs. Numba

Single-threaded Numba (`@njit`) on the GBM simulator showed **no meaningful improvement** over vectorized NumPy (0.9x-1.0x) — NumPy's `cumsum`/`exp` already execute as optimized, allocation-light C routines, so a Numba loop calling `np.random.normal` as a scalar function 252 million times (at $N=1M$) gains nothing from JIT compilation alone.

Parallelizing across the independent-path dimension with `numba.prange` gave a genuine, reproducible **$\sim 9x$ speedup** ($37.216s \pm 0.711s \rightarrow 4.130s \pm 0.070s$, mean $\pm$ std over 5 repeated

trials), despite 28 logical threads being available — a sub-linear but real result, most plausibly explained by the workload being memory-bandwidth-bound (writing ~2GB of path data) rather than purely compute-bound.

A benchmarking lesson: an early single-run measurement gave a misleadingly low 1.7x speedup, later shown to be machine noise (a subsequent full-notebook rerun gave a consistent 8.5–9x across four independent measurements). This motivated switching to repeated-trial benchmarking (mean $\pm$ std over 5 runs) as the standard methodology for every performance claim in this document, rather than trusting any single wall-clock measurement.

Longstaff-Schwartz optimization

Profiling (`cProfile`) revealed the actual bottleneck breakdown for the LS pricer at $N=100,000$, $n_steps=252$ (total 7.618s): **path simulation, 46%; regression (`lstsq`), 21%;** remaining loop overhead, ~33%. This corrected an initial assumption that regression would dominate — profiling before optimizing changed the priority order.

Optimizations applied: (1) swapped the plain simulator for the Numba-parallel version from the path-simulation work above; (2) replaced (`np.linalg.lstsq`) with a direct normal-equations solve (`np.linalg.solve` on the small 3×3 $A^T A$ system), avoiding (`lstsq`)'s more general SVD-based solver. Result: **1.63x speedup** (6.525s $\rightarrow$ 4.011s), with price output unchanged within CI (6.0207 ± 0.0431 before, 6.0795 ± 0.0441 after).

This falls well short of the ~9x seen for the simulator in isolation — a direct illustration of **Amdahl's Law**: since the optimized components (simulation + regression) accounted for roughly two-thirds of total runtime, even a large speedup on those parts is capped by the unoptimized remainder, which becomes the new proportional bottleneck.

8. Automated Test Suite

22 automated tests (`pytest`) cover every validation check described above: Black-Scholes correctness (including the off-ATM regression case), MC-to-BS convergence for all four variance reduction methods, Greeks agreement against closed form, exotic payoff validations (geometric closed-form match, Jensen's inequality direction, in/out barrier parity, extreme limits), and American option checks (put premium existence, call-put early-exercise equivalence, LS-vs-binomial agreement, and consistency between the original and performance-optimized LS implementations).

A test-design lesson: two tests initially failed intermittently — one comparing American and European call prices, one checking barrier in/out parity — both because their tolerance was based on a single estimate's standard error, when the quantity being tested was actually a *difference* between two or three independent Monte Carlo estimates. The statistically correct approach combines standard errors in quadrature ($\sqrt{se_1^2 + se_2^2 + \dots}$).

...)) before setting a tolerance; both tests were corrected accordingly and pass reliably at 22/22.

Full suite runtime: ~2 minutes (dominated by tests using N=100,000–200,000 for tight confidence intervals).

9. Summary of Key Numbers

Result	Value
Antithetic variance reduction	1.93x
Sobol RQMC speedup (1D, vs. naive)	971x
Pathwise vs. LR Delta CI width ratio	~2.5x
Pathwise vs. LR Vega CI width ratio	~3.6x
American put early-exercise premium	~0.47 (6.06 vs. 5.59)
LS vs. binomial agreement (put)	within 0.9 SE
Numba parallel speedup (path simulation)	~9x
LS pipeline speedup (post-profiling)	1.63x
Automated test suite	22/22 passing